

MOD-SR: Unifying Multimodal Learning and Direct Optimization with Gradient-Guided Diffusion Model for Symbolic Regression

Chuyang Xiang^{*1} Yichen Wei^{*1} Junchi Yan¹

Abstract

Symbolic regression (SR) aims to discover interpretable mathematical expressions from observed data. While recent generative approaches have shown promise in treating SR as machine translation or multimodal learning tasks using NN methods, they suffer from uncontrollable generation process and training-evaluation misalignment. The training objectives (average cross-entropy loss on a token level across the distribution of historical data) differ from the evaluation metric (fitting error for every test data / complexity), necessitating extensive heuristic post-processing. On the other hand, direct optimization methods suffer from an exponential slowdown as the dimensionality increases, non-differentiability and local optima traps. We propose **MOD-SR**, unifying multimodal distribution learning during training with direct optimization at inference time. This is achieved by modeling the task as $p(x_0 | \mathcal{D}, y^*)$ and employing gradient-guided diffusion in embedding space, enhanced by contrastive learning and representation alignment. Furthermore, we introduce DFEX, a fixed-depth tree relaxation method that ensures differentiability for effective gradient guidance during inference. Experiments demonstrate that MOD-SR achieves superior performance on diverse benchmarks through a unified framework integrating distribution learning and optimization. Our code is available at <https://github.com/KROX777/MOD-SR>.

^{*}Equal contribution This work was supported by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China, JYB2025XDXM411. We also acknowledge support from the Zhiyuan Innovative Research Center through the Zhiyuan Future Scholars Program (ZIRC2025-22).
¹School of Artificial Intelligence, Shanghai Jiao Tong University. Correspondence to: Junchi Yan <yanjunchi@sjtu.edu.cn>.

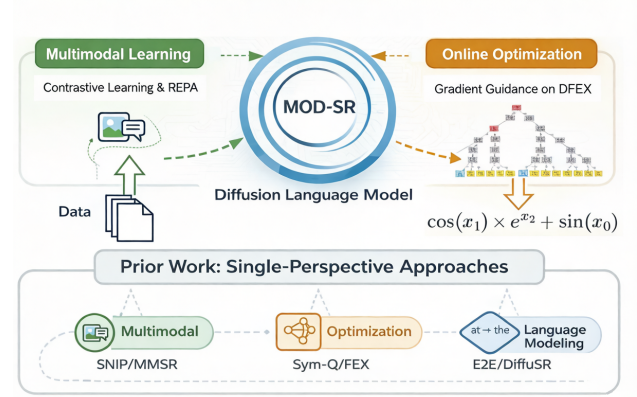


Figure 1. Our MOD-SR adopts gradient-guided diffusion model, being the first unified model integrating multimodal learning and direct optimization for symbolic regression.

1. Introduction

Symbolic regression (SR) seeks to discover explicit mathematical expressions that describe observed data, offering interpretability and scientific insight beyond black-box predictive models (Vafa et al., 2025). This problem is fundamental in scientific discovery (Udrescu & Tegmark, 2020; Feng et al., 2025), enabling researchers to uncover underlying physical laws (Hu et al., 2025) and understand causal relationships. It also has applications in analytical PDE solver (Oh et al., 2023; Cao et al., 2025). Unlike traditional regression that fits parameters within a fixed form, symbolic regression must search over a combinatorial space of mathematical structures, making it inherently a combinatorial optimization problem.

Direct optimization methods, such as genetic programming (GP) (Fang et al., 2023; Raghav et al., 2024) and reinforcement learning (RL) approaches (Tenachi et al., 2023), directly optimize the fitting objective by searching over expression structures. Although there exist some methods speeding up the search process through NN (Yu et al., 2025b; Kamienny et al., 2023), this line of work still suffers from an exponential slowdown as the dimensionality of independent variables increases.

Deep generative approaches have shown promise by learning to generate expressions directly from numerical obser-

Table 1. Comparison of different symbolic regression methods.

METHOD	PERSPECTIVE	DISTRIBUTION LEARNING	OPTIMIZATION
E2E (KAMIENNY ET AL., 2022)	AR (TRANSLATION)	TRANSFORMER	BEAM SEARCH
MMSR (LI ET AL., 2025B)	AR (MULTIMODAL)	TRANSFORMER	BEAM SEARCH
SNIP (MEIDANI ET AL., 2024)	AR (MULTIMODAL)	TRANSFORMER + CL	HEURISTIC (GP)
FEX (LIANG & YANG, 2025)	MDP (OPTIMIZATION)	/	RL
SYM-Q (TIAN ET AL., 2024)	MDP (OPTIMIZATION)	Q-LEARNING	RL (REINFORCE)
DBSR (BASTIANI ET AL., 2025)	MDP (OPTIMIZATION)	/	RL (GRPO)
METASYMNET (LI ET AL., 2025A)	EQL (OPTIMIZATION)	/	NOVEL NN
SYMBOLIC DIFFUSION (TYMKOW ET AL., 2025)	BAYESIAN (TRANSLATION)	DISCRETE DIFFUSION	/
DIFFUSR (HAN ET AL., 2025)	BAYESIAN (TRANSLATION)	DIFFUSIONLM	HEURISTIC (GP)
GENSR (LI ET AL., 2026)	BAYESIAN (MULTIMODAL)	CONDITIONAL VAE	HEURISTIC (CMA-ES)
MOD-SR (OURS)	BAYESIAN (MULTIMODAL + OPTIMIZATION)	GRADIENT-GUIDED DIFFUSIONLM + REPA/CL	

AR: AUTOREGRESSIVE, MDP: MARKOV DECISION PROCESS, EQL: EQUATION LEARNER, CL: CONTRASTIVE LEARNING, DIFFUSIONLM: DIFFUSION LANGUAGE MODEL (LI ET AL., 2022), REPA: REPRESENTATION ALIGNMENT.

uations, circumventing the long trial-and-error process fundamentally. Since an expression tree can be serialized into a token sequence (e.g., by preorder traversal), autoregressive (AR) models are a natural choice for SR, and early methods indeed adopted an end-to-end AR pattern, modeling the task as a machine translation problem (Kamienny et al., 2022; Biggio et al., 2021). Later, researchers realized the inherent weakness of this machine translation formulation and began interpreting this task as a multimodal task (Li et al., 2025b). Some still followed the AR pattern (Meidani et al., 2024), others tried new ways like VAEs (Li et al., 2026) and diffusion language models (Han et al., 2025).

However, all existing generative methods suffer from the following flaws: 1) Training-evaluation misalignment. The training objectives (cross-entropy loss on a token level) are fundamentally different from the evaluation metric we care about: fitting error on data / complexity. This misalignment makes these methods rely heavily on heuristic post-processing (GP (Meidani et al., 2024), CMA-ES (Li et al., 2026), etc.). 2) Uncontrollable generation process. Although AR generation is natural for SR, its discrete left-to-right token sampling breaks the gradient path from the evaluation metric to the generation trajectory, making direct inference-time metric guidance difficult. As a result, it is hard to inject specific priors or objectives into the inference process of these methods without re-training the model, and the model itself cannot obtain actual feedback from its guess. (Most existing methods inject physics prior into the architecture itself (Simon Keren et al., 2023; Ying et al., 2025)) There exists no unified model that simultaneously gives consideration to the optimization essence of this task and NNs’ strong capabilities of learning rich prior distributions.

Thus, we propose **MOD-SR**, unifying multimodal distribution learning with direct optimization with one single

model:

- We formulate symbolic regression as multimodal conditional distribution learning using continuous diffusion in embedding space, leveraging a frozen numerical encoder, enhanced by representation alignment during training with a symbolic encoder from the same contrastive framework.
- We develop gradient guidance that directly optimizes the fitting objective during sampling, which not only mitigates the training-evaluation misalignment, but also makes the generation process controllable and easy to optimize other objectives, such as complexity.
- We proposed the Differentiable Finite Expression Tree (DFEX) for encoding expressions in a fixed tree, which can be optimized by gradient descent through relaxation. This not only helps the gradient guidance process, but also provides a new way of differentiable symbolic expression optimization.
- Experiments show our method achieves superior performance on diverse benchmarks by unifying distribution learning and optimization. The symbolic embedding space learned by our model also demonstrates higher-quality representations than existing methods.

2. Related Work

Symbolic Regression as an Optimization Task To mitigate the exponential slowdown as the dimensionality increases inherent in traditional methods like Genetic Programming (GP), recent methods integrate neural guidance: neural-seeded GP (Mundhenk et al., 2021), transformer-guided Monte-carlo tree search (Yu et al., 2025b; Shojaee

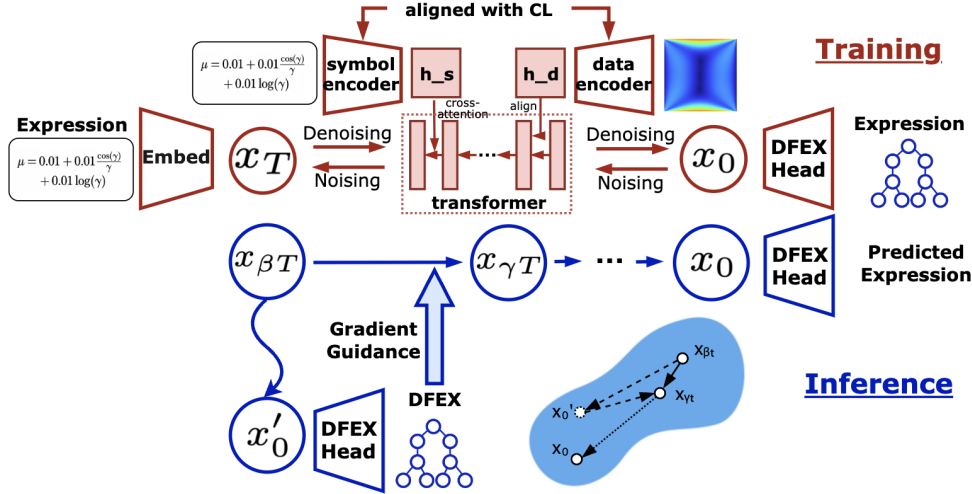


Figure 2. Overview of our method: multimodal encoders pre-trained with contrastive learning condition the diffusion model, representation alignment with the symbolic encoder from the same contrastive framework during the training process, DFEX relaxes trees for differentiable evaluation, enabling gradient guidance to directly optimize objectives during sampling.

et al., 2023). RL methods model expression construction as a Markov Decision Process (MDP), using policy gradients (Liang & Yang, 2025) or offline RL with human-in-the-loop co-design (Tian et al., 2024). LLM is also used to do heuristic search (Shojaee et al., 2025). To address the non-differentiability of discrete expression structures, another line of work introduces differentiable optimization techniques for SR. Differentiable Genetic Programming (DGP) (Zeng et al., 2023) introduces a differentiable symbolic tree, where discrete operator/variable choices are relaxed, yet fails to handle coefficients. MetaSymNet (Li et al., 2025a) follows a related direction based on softmax-like meta-functions and adaptive structure optimization, but adopting the Equation Learner (EQL) pattern as it directly trains a newly-designed NN on datasets. Other approaches like DDSR-NN (Lu et al., 2025) mix discrete and continuous optimization, incorporating gate masks and rule masks. These optimization-centric methods excel at directly minimizing the target fitting error but often scale poorly to high-dimensional expressions and struggle to leverage large-scale pre-training on symbolic data.

Symbolic Regression as a Multimodal Distribution Learning Task Early approaches often framed symbolic regression (SR) as a sequence-to-sequence translation task (Kamienny et al., 2022), a formulation that carries theoretical limitations. Recent studies have increasingly recognized SR as an inherently *multimodal problem*, in which numerical data and symbolic expressions constitute fundamentally distinct modalities. For instance, SNIP (Meidani et al., 2024) employs contrastive learning to align dual encoders in a shared latent space, while MMSR (Li et al., 2025b) incorporates feature fusion during expression generation. Both methods, however, follow an end-to-end learning

scheme. Further work adopts a Bayesian perspective. Notably, DiffuSR (Han et al., 2025) applies a diffusion model to SR, directly adapting the DiffusionLM (Li et al., 2022) framework; yet it does not adopt a multimodal perspective and also relies on GP for post-processing. GenSR (Li et al., 2026) uses a dual-branch conditional VAE to model $p(f|\mathcal{D})$, emphasizing numerical behavior. However, it still requires CMA-ES post-processing. These works collectively reveal that the learning of a prior distribution and the optimization of the final expression remain fundamentally separate.

Gradient Guidance in Diffusion Models Gradient guidance in diffusion models has been proven effective in controllable image generation (Black et al., 2024), combinatorial optimization (Li et al., 2023; 2024) and PDE inverse problem solving (Yao et al., 2026; Lin et al., 2026). By steering generation toward desired properties at test time by modifying the score function, gradient guidance allows for incorporating task-specific objectives without retraining. From a Bayesian perspective, it can also be viewed as refining a learned prior toward an objective-conditioned posterior during inference. An early representative is classifier guidance (Dhariwal & Nichol, 2021). Recent work on the optimization perspective of diffusion models (Guo et al., 2024) highlights that naive guidance $\nabla_{x_t} \mathcal{L}(x_t)$ often leads to off-manifold samples. Instead, look-ahead guidance based on conditional expectation $\mathbb{E}[x_0|x_t]$ effectively preserves data structure while optimizing the objective.

3. Methodology

An overview of our method is shown in Figure 2. Table 1 compares MOD-SR with existing symbolic regression approaches, which shows how we unify the two perspectives.

3.1. Problem Formulation

Given a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ where $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$, symbolic regression seeks to find a mathematical expression f such that $y_i \approx f(x_i)$ for all i . We represent expressions as token sequences $e = [e_1, \dots, e_L]$ from a vocabulary $\mathcal{V}$ containing operators $(+, -, \times, \div)$, functions $(\sin, \cos, \exp, \log, \dots)$, variables $(x_1, \dots, x_d)$, and constants.

Following recent multimodal perspectives (Li et al., 2025b; Meidani et al., 2024), we recognize that numerical data $\mathcal{D}$ and symbolic expressions e represent distinct modalities—analogue to images and text in vision-language tasks. Unlike machine translation, where source and target share linguistic structure, data points (continuous, unordered sets) and expressions (discrete, tree-structured) are fundamentally different. We model the conditional distribution across modalities: $p(x_0 | c, y^*)$, $c = \text{Enc}_{\text{num}}(\mathcal{D})$ where x_0 is the latent representation of the expression in embedding space, $\mathcal{D}$ represents the observed numerical data, and y^* is the optimal objective score given $\mathcal{D}$. c is the cross-modal condition obtained from a frozen numerical encoder. We’ll further elaborate on the meaning of y^* conditioning in Section 3.5.

3.2. Diffusion Model Architecture

We stress that the motivation for introducing diffusion is that our goal is not only distribution learning, but also *direct inference-time optimization*: the fitting metric should guide the generation trajectory for each test instance. This requires a continuous and differentiable path from the objective to the latent expression, which discrete AR token sampling does not provide. Continuous diffusion in embedding space offers such a path and therefore makes objective-aware generation possible (Section 3.5).

Since learning to output an expression is, first of all, a grammar-learning task, we adopt continuous diffusion in embedding space, following DiffusionLM (Li et al., 2022). This formulation enables the model to learn the grammar itself instead of adding additional constraints (Mežnar et al., 2023), which will make the training very hard.

Token Embedding Space. Let $\mathcal{V}$ be the vocabulary of mathematical symbols. For an expression e of length L , we obtain the initial embedding $x_0 = \mathcal{E}(e) \in \mathbb{R}^{L \times d_e}$ where $\mathcal{E}$ is a learnable embedding layer. Following DiffusionLM (Li et al., 2022), we tie the weights of the embedding layer $\mathcal{E}$ with the output layer to share parameters, enabling efficient rounding from continuous embeddings back to discrete tokens via argmax on the logits. Unlike approaches that use pooled global representations such as SNIP (Meidani et al., 2024), we operate in a token-level embedding space. This allows our model to leverage language modeling principles to preserve both the syntactic structure (grammar) and the

precise symbolic semantics of the expression, which are critical for accurate recovery and gradient-based optimization. We will also showcase the superiority of this token-wise embedding space in Section 5.

Forward Process. The forward process initializes with $x_0 = \mathcal{E}(e)$, a random variable drawn from the distribution of encoded expressions. It introduces noise via an Ornstein-Uhlenbeck process:

$$dx_t = -\frac{1}{2}q(t)x_t dt + \sqrt{q(t)}dW_t, \quad (1)$$

where $(W_t)_{t \geq 0}$ is a Wiener process and $q(t)$ is the non-decreasing noise schedule. x_t represents the noise-corrupted data distribution at time t . The conditional distribution $x_t|x_0$ is Gaussian $\mathcal{N}(\alpha(t)x_0, h(t)I)$ with $\alpha(t) = \exp(-\int_0^t \frac{1}{2}q(s)ds)$ and $h(t) = 1 - \alpha^2(t)$.

Reverse Process (Score Matching). Reversing the forward process allows reconstructing the original distribution from pure noise. The backward SDE is given by:

$$dx_t^{\leftarrow} = \left[\frac{1}{2}x_t^{\leftarrow} + \nabla \log p_{T-t}(x_t^{\leftarrow}) \right] dt + d\bar{W}_t, \quad (2)$$

where $(\bar{W}_t)_{t \geq 0}$ is an independent Wiener process. The core task is to learn the score function $\nabla \log p_t(x_t)$ using a score network $s_\theta(x_t, t, c)$ conditioned on the numerical embedding c . We train s_θ by minimizing the score matching loss:

$$\mathcal{L}_{\text{score}} = \mathbb{E}_{t, x_0, x_t} [\|\nabla_{x_t} \log \phi_t(x_t|x_0) - s_\theta(x_t, t, c)\|^2] \quad (3)$$

where $\phi_t(x_t|x_0)$ is the Gaussian transition kernel of the forward process. The generator uses cross-attention to condition on c from the frozen numerical encoder. See Appendix B for architectural details.

Reconstruction Loss. To ensure generated embeddings map back to the correct tokens, we add:

$$\mathcal{L}_{\text{recon}} = -\sum_{l=1}^L \log p_\theta(e_l|\hat{x}_0) \quad (4)$$

3.3. Multimodal Treatment

Cross-Modal Contrastive Pre-training. Drawing from the experience of Stable Diffusion (Rombach et al., 2022), we recognize the necessity of pre-training a numerical encoder separately, rather than training everything jointly as in Symbolic Diffusion, which yields poor results. Following SNIP (Meidani et al., 2024), we pre-train dual encoders to align numerical and symbolic modalities in a shared latent space to enhance the diffusion model using contrastive learning (CL).

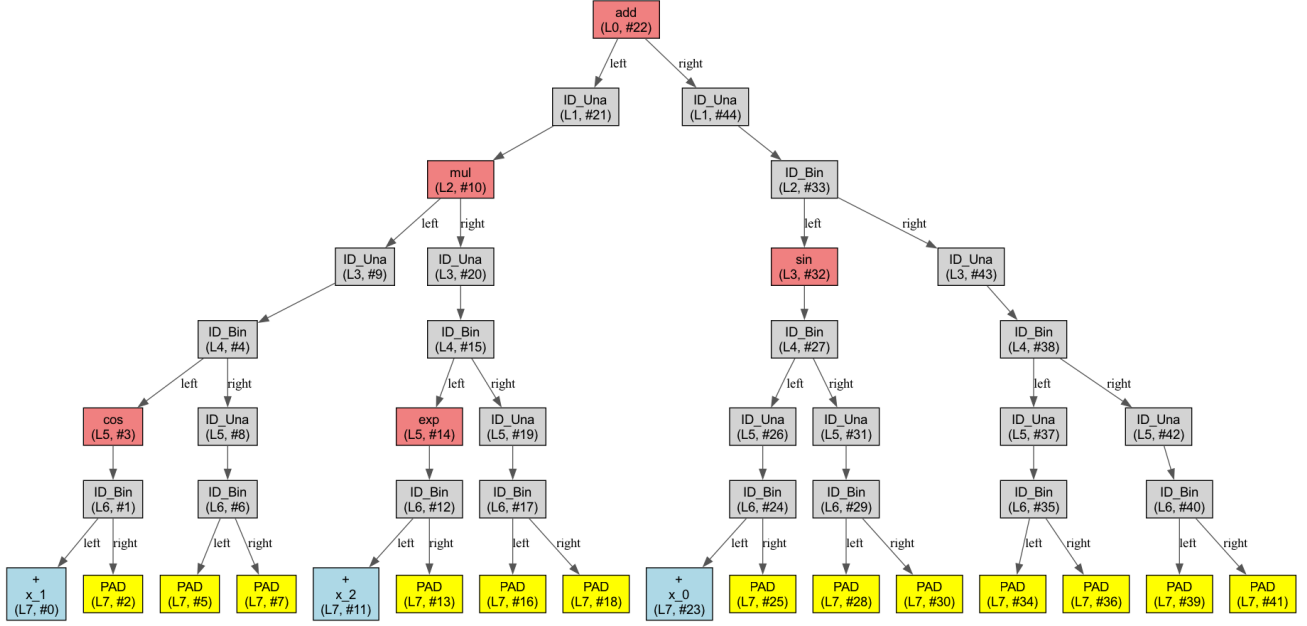


Figure 3. Example of DFEX tree structure of depth 3 encoding expression $y = \cos(x_1) \times e^{x_2} + \sin(x_0)$. The leaves are set to be the variables or constants, each occupying two leaves for one subject, while the nodes and root are set to be the operations. The Unary and Binary layers restrict the type of operations.

Frozen Numerical Conditioning. We use the frozen pre-trained numerical encoder to provide stable conditioning $c \in \mathbb{R}^{L_c \times d_c}$. We extract features before the final pooling to preserve sequential information for cross-attention with the denoiser, unlike the pooled representation used in standard SNIP. This multimodal conditioning provides rich numerical context without introducing training instability.

Representation Alignment. To further enhance generation quality, we align the intermediate representations of our diffusion model with a frozen symbolic encoder, drawn from the same CL framework mentioned above. This approach is inspired by REPA (Yu et al., 2025a), originally developed for Diffusion Transformer (Peebles & Xie, 2023) in image synthesis. The encoder acts as a teacher, projecting expressions into a high-quality latent space. During training at layer l , we extract student features $h_{\text{student}}^{(l)}$ and align them with teacher features h_{teacher} . We also introduce a signal-to-noise weighting to the original REPA loss to address performance degradation observed in initial implementations:

$$\mathcal{L}_{\text{align}} = \sqrt{\alpha_t} \left(1 - \frac{1}{L} \sum_{i=1}^L \frac{h_{\text{teacher}}^{(i)} \cdot \text{MLP}(h_{\text{student}}^{(i)})}{\|h_{\text{teacher}}^{(i)}\| \|\text{MLP}(h_{\text{student}}^{(i)})\|} \right) \quad (5)$$

This alignment encourages the model to produce embeddings that are consistent with high-quality representations. Altogether, the training objective is:

$$\mathcal{L} = \mathcal{L}_{\text{score}} + \lambda_{\text{recon}} \mathcal{L}_{\text{recon}} + \lambda_{\text{align}} \mathcal{L}_{\text{align}} \quad (6)$$

3.4. Differentiable Finite Expression Tree

We introduce the Differentiable Finite Expression Tree (DFEX), a fixed-depth tree relaxation method that allows $\mathcal{L}_{\text{fit}}$ to be a differentiable signal during inference (Section 3.5). DFEX involves two parts: the fixed-depth tree structure design and relaxation method. The tree design in DFEX follows the FEX framework (Liang & Yang, 2025), originally developed for PDE solving with RL (Huynh et al., 2026):

DFEX is structured with a depth of d , consisting of alternating binary and unary layers. Binary layers (at even depths) accommodate operations such as addition and multiplication, as well as identity operations (`<ID_Binary>`) for masking unused branches. Unary layers (at odd depths) include functions like $\sin$ and $\log$, as well as identity operations (`<ID_Unary>`) for pass-through, along with placeholders (`<PAD>`) for constants and variables at the leaf nodes. This design makes it convenient to modify the expression structure during gradient optimization.

Note that by a depth of d , we mean d unary layers, d binary layers, and a layer of leaf. The length of the sequence is therefore $2^{d+2} - 1$, enjoying sufficient expressivity.

Essentially, this fixed-depth tree is a binary-like tree structure. In practice, we serialize it into a sequence where each index in the sequence corresponds to a fixed node in the tree, making it easy to be predicted by Transformer-based architectures. An example tree structure is shown in Figure 3.

Upon the fixed-depth tree, DFEX computes token probabilities via softmax. Although this shares a similar relaxation

spirit with DGP (Zeng et al., 2023) and MetaSymNet (Li et al., 2025a), our purpose is different: DFEX is an auxiliary interface for inference-time token-level gradient routing inside the diffusion process, rather than a standalone framework that directly optimizes the whole symbolic model on the target dataset. Besides, DGP supports skeleton-only refinement without giving consideration to coefficients.

For constants, we use 2-token encoding: mantissa m and exponent e , where the relaxed value is computed as:

$$c = \left(\sum_i p_{m,i} \cdot i \right) \times 10^{(\sum_j p_{e,j} \cdot j)} \quad (7)$$

where $p_{m,i}$ and $p_{e,j}$ are softmax probabilities over mantissa and exponent tokens. For variables, the relaxed value combines sign and variable probabilities:

$$v = (p_+ - p_-) \times \sum_k p_{x_k} \cdot x_k \quad (8)$$

More details can be found in Appendix F.

DFEXHead. In practice, instead of directly training the diffusion model on DFEX sequence, we train an additional DFEXHead to interpret the embedding into DFEX logits, leaving the denoising process still on the embedding space of simple infix expressions. We provide a detailed rationale for this in Appendix A.

3.5. Gradient Guidance for Direct Optimization Objective

Our method introduces gradient guidance at inference time. Gradients derived from the desired objective y^* steer the sampling trajectory, effectively performing a posterior refinement within the diffusion manifold—a Bayesian perspective that aligns with GenSR (Li et al., 2026).

Gradient Guidance of Look-Ahead Loss. During inference we sample from the objective-conditioned distribution $p_\theta(x_0 | x_t, \mathcal{D}, y^*)$. Let $s(e; \mathcal{D})$ denote the measured objective of expression e on dataset $\mathcal{D}$. We want $s(e; \mathcal{D})$ to match the target score y^* , so we define a look-ahead loss

$$\mathcal{L}_{\text{fit}}(e; \mathcal{D}, y^*) = \varphi(s(e; \mathcal{D}), y^*) \quad (9)$$

for some discrepancy function φ (MSE, complexity, etc.). Following the look-ahead guidance framework (Guo et al., 2024), we approximate the conditional score $\nabla_{x_t} \log p_\theta(y^* | x_t, \mathcal{D})$ by differentiating the objective on the predicted clean sample $\hat{x}_0(x_t)$:

$$\mathbf{G}_{\text{loss}}(x_t, t) = -w(\hat{t}) \nabla_{x_t} \mathcal{L}_{\text{fit}}(\hat{x}_0(x_t); \mathcal{D}, y^*), \quad (10)$$

where $w(\hat{t})$ is the schedule from Appendix B. The look-ahead estimate $\hat{x}_0(x_t) = \mathbb{E}[x_0 | x_t]$ is computed via

Tweedie’s formula:

$$\hat{x}_0(x_t) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t, c) \right). \quad (11)$$

Gradient Computation via Chain Rule. Applying the chain rule yields

$$\nabla_{x_t} \mathcal{L}_{\text{fit}}(\hat{x}_0; \mathcal{D}, y^*) = \frac{\partial \mathcal{L}_{\text{fit}}}{\partial \hat{x}_0} \frac{\partial \hat{x}_0}{\partial x_t}, \quad (12)$$

The Jacobian $\frac{\partial \hat{x}_0}{\partial x_t}$ can be easily evaluated by backpropagating through the denoiser. The hard part is the remaining term $\partial \mathcal{L}_{\text{fit}} / \partial \hat{x}_0$. Using the above-mentioned differentiable DFEX tree. We can obtain it from the DFEXHead. For clarity we express $\partial \mathcal{L}_{\text{fit}} / \partial \hat{x}_0$ via intermediate variables:

$$\frac{\partial \mathcal{L}_{\text{fit}}}{\partial \hat{x}_0} = \frac{\partial \mathcal{L}_{\text{fit}}}{\partial s} \frac{\partial s}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial (\mathbf{T}, \mathbf{A})} \frac{\partial (\mathbf{T}, \mathbf{A})}{\partial \hat{x}_0}, \quad (13)$$

where $\mathbf{h}$ are DFEX evaluations on $\mathcal{D}$, $\mathbf{T}/\mathbf{A}$ are tree tokens/attributes, and the first factor depends on y^* through $\mathcal{L}_{\text{fit}}$. Note that during guidance we freeze the global tree and only keep the selected subtree’s logits “soft” (top- k) while clamping the rest to their top-1 tokens; this *random subtree masking* technique considerably lowers the cost while preserving effects.

Guided DDIM Update. Incorporating the guidance term into DDIM (Song et al., 2021) gives

$$x_{t-\Delta t} = \sqrt{\bar{\alpha}_{t-\Delta t}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-\Delta t}} \epsilon_t - \gamma \mathbf{G}_{\text{loss}}(x_t, t), \quad (14)$$

so each step follows the conditional distribution $p_\theta(x_{t-\Delta t} | \mathcal{D}, y^*)$ up to the guidance coefficient γ .

3.6. Perturbation for Expression Search

With gradient guidance, our framework enables a native post-processing method through perturbation through noise injection for thorough expression search.

Perturbation via Reverse SDE. Starting from x_t we view the sampler as evolving under the reverse-time SDE

$$dx = [f(t)x - g^2(t) \nabla_x \log p_t(x | \mathcal{D}, y^*)] dt + g(t) d\bar{W}_t, \quad (15)$$

with drift/variance (f, g) dictated by the scheduler. We then take a single Euler–Maruyama step whose stochastic term matches the forward process. Practically we draw

$$x_{t+\delta} \sim q_\theta(x_{t+\delta} | x_t) = \mathcal{N}(\alpha(\delta) x_t, [1 - \alpha^2(\delta)] I), \quad (16)$$

where $\delta = \lfloor \alpha_{\text{rw}} T \rfloor$ and α_{rw} is the rewrite ratio.

Constrained Reconstruction with y^* Guidance. The noisy latent $x_{t+\delta}$ is immediately projected back using a short deterministic DDIM trajectory ($\tau = 10$ steps) that reuses the y^* -conditioned gradient. Each perturb–reconstruct cycle, therefore, performs a discrete reverse-SDE hop followed by objective-guided refinement.

Table 2. Performance comparison of different SR methods *without* post-processing on benchmark datasets. The table shows R^2 scores (higher is better) and average number of nodes ($\mathcal{C}$, lower is better) for each method.

Group	Dataset	MOD-SR		DiffuSR*		SNIP		E2E		SymbolicGPT [†]	
		R^2	$\mathcal{C}$	R^2	$\mathcal{C}$	R^2	$\mathcal{C}$	R^2	$\mathcal{C}$	R^2	$\mathcal{C}$
Standard	Nguyen	0.984	16.6	0.986	8.6	0.938	15.2	0.671	19.3	0.671	21.6
	Keijzer	0.970	17.3	-	-	0.802	17.4	0.861	19.5	0.603	24.5
	Koza	1.000	10.0	-	-	1.000	9.5	1.000	19.0	0.661	29.2
	Constant	0.983	15.5	0.964	10.6	0.896	12.1	0.755	16.6	0.702	38.5
	Livermore	0.989	16.6	0.943	9.4	0.947	14.0	0.720	20.7	0.563	41.2
	Vladislavleva	0.907	21.1	-	-	0.769	20.6	0.743	33.8	0.541	36.6
	R	0.985	15.7	-	-	0.774	16.7	0.991	26.0	0.704	25.2
	Jin	0.995	18.5	0.797	14.5	0.964	16.7	0.832	15.7	0.772	36.9
SRBench	Feynman	0.788	16.4	-	-	0.616	18.1	0.735	22.0	0.538	26.8
	Strogatz	0.970	15.8	-	-	0.854	21.3	0.754	31.5	0.523	32.6

*: DiffuSR is not open-sourced, all of its test data is directly taken from the original paper (also top-20 sampling). They only test on 4 benchmarks, so there are blanks. †: We don't include Symbolic Diffusion (Tymkow et al., 2025) since it's reported to have equivalent performance to SymbolicGPT (Mojtaba Valipour, 2021), which is inferior in our experiments.

4. Experiments

4.1. Experimental Setup

For the training dataset, we follow the same synthetic data generation procedure as E2E (Kamienny et al., 2022) and SNIP (Meidani et al., 2024) (Appendix B). For evaluation, we compare MOD-SR against baselines using multiple standard datasets (Li et al., 2025b). Note that we could not obtain all black-box datasets from SRBench (La Cava et al., 2021) due to the discontinued maintenance of the PMLB repository¹; thus, our SRBench evaluation is limited to the Feynman and Strogatz datasets.

We first test MOD-SR without any post-processing methods except BFGS constant optimization to evaluate the raw generation quality. Then we evaluate MOD-SR with post-processing methods to compare with existing methods fairly. After that, we conduct some ablation studies as well as inference time comparison. All hyperparameters can be found in Appendix B.

Table 3. Performance comparison of different post-processing methods.

Dataset	MOD-SR (Pert. w/ GG)		SNIP (LSO)		DiffuSR (GP)	
	R^2	$\mathcal{C}$	R^2	$\mathcal{C}$	R^2	$\mathcal{C}$
Nguyen	0.999	15.5	0.945	23.2	0.998	16.7
Keijzer	0.983	16.8	0.819	24.0	-	-
Koza	1.000	9.5	1.000	9.5	-	-
Const.	0.995	18.1	0.995	18.2	0.999	19.1
Liver.	0.997	15.5	0.950	19.5	0.995	20.5
Vlad.	0.950	18.5	0.860	34.0	-	-
R	1.000	15.7	0.999	21.7	-	-
Jin	1.000	15.7	0.977	25.6	0.973	28.2

¹<https://github.com/EpistasisLab/pmlb>

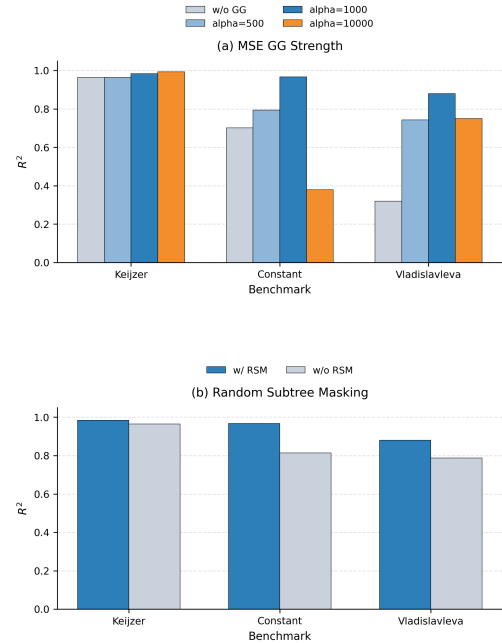


Figure 4. Ablation study on gradient guidance strength and random subtree masking.

4.2. Evaluation Without Post-Processing

Table 2 compares MOD-SR with baseline methods, bypassing any post-processing methods except BFGS constant optimization. (For us, we bypass the perturbation mechanism.) All tests use top-20 sampling for diffusion models and beam size=20 for AR models. Note that our diffusion model begins predicting valid token sequences from $T = 556$ on an average level (DDIM sampling), which is similar to the result of DiffuSR (Han et al., 2025).

Table 4. Comparison of different guidance objectives on the shared benchmark suite. MSE guidance is evaluated by fitting performance, while length guidance is evaluated by both fitting performance and complexity.

Dataset	w/o GG		MSE GG		Length GG	
	R^2	$\mathcal{C}$	R^2	$\mathcal{C}$	R^2	$\mathcal{C}$
Nguyen	0.979	16.2	0.984	16.6	0.983	16.1
Keijzer	0.970	17.3	0.970	17.3	0.970	15.5
Koza	1.000	8.5	1.000	8.5	1.000	10.5
Constant	0.929	18.6	0.983	15.5	0.937	18.0
Livermore	0.989	16.6	0.989	16.6	0.989	14.9
Vladislavleva	0.896	19.3	0.907	21.1	0.925	19.3
R	0.985	15.7	0.985	15.7	0.992	15.0
Jin	0.995	19.5	0.995	18.5	0.995	17.3
Avg	0.968	17.0	0.976	16.9	0.974	16.0

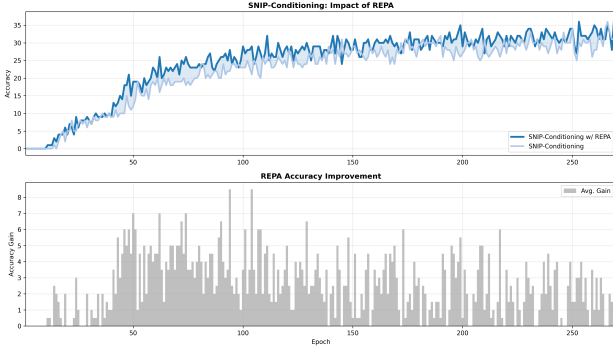


Figure 5. Effect of Representation Alignment. Removing representation alignment leads to slower training convergence and degraded final performance.

4.3. Evaluation on Post-Processing Method

Table 3 shows that the native perturbation-based post-processing method of MOD-SR outperforms GP and Latent Space Optimization (LSO) across benchmarks. This demonstrates the effectiveness of our proposed method. We discuss the runtime control in Appendix B.4.

4.4. Ablation Study

Representation Alignment. For the ablation study, we bypass the effect of representation alignment on training dynamics and final performance. As illustrated in Fig. 5, without it, the diffusion model exhibits slower convergence during training. Additionally, the final performance is noticeably lower.

Gradient Guidance. We ablate gradient guidance from three aspects in Table 4 and Figure 4. First, comparing guidance objectives: MSE guidance improves average R^2 from 0.968 to 0.976, while length guidance preserves accuracy ($R^2 = 0.974$) but reduces complexity from 17.0 to 16.0, showing the same machinery can steer toward different inference-time objectives. Second, Figure 4 shows random subtree masking (RSM) consistently outperforms full-tree

optimization on hard cases, validating that localized updates are cheaper yet more effective. Note that we use the same seed when running the experiment for gradient guidance to enable meaningful comparison.

4.5. Inference Runtime Comparison

To isolate the raw inference cost, we additionally run each model on a single instance with beam size 1 for autoregressive models and top-1 sampling for MOD-SR, then average the runtime over all benchmarks. We also report SymQ with REINFORCE-based online optimization as a reference. The results are shown in Table 5. Gradient guidance slows inference because it involves optimization, but it remains substantially cheaper than online RL and can be beneficial in the long run once post-processing is enabled (Table 3).

Table 5. Single-instance inference runtime comparison. AR models use beam size 1, while MOD-SR uses top-1 sampling. Each runtime is averaged over all benchmarks.

Method	Runtime
SNIP	1.13s
E2E	1.41s
MOD-SR (w/o GG)	2.80s
MOD-SR (w/ GG)	18.44s
SymQ (w/ REINFORCE)	192.60s

5. Further Discussion on Symbolic Latent Space

A foundational design choice in MOD-SR is operating in a token-wise embedding space rather than following the compression paradigm of prior multimodal SR methods such as SNIP (Meidani et al., 2024), which pools per-token representations into a single global vector. We argue that this compression is detrimental for generative SR, and substantiate this by comparing three spaces: (1) **SNIP pooled** (mean-pooled global vector), (2) **SNIP token-wise** (same encoder, no pooling), and (3) **MOD-SR embedder** (jointly trained with the denoiser).

4x3 latent comparison: per-metric uniform sample (5k each row)

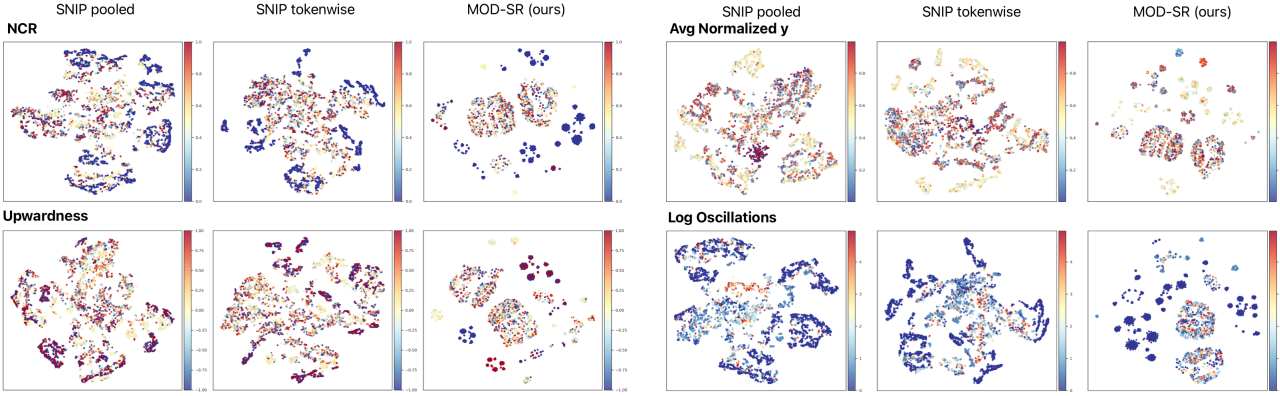


Figure 6. t-SNE visualization comparing three representation spaces: SNIP pooled, SNIP token-wise, and MOD-SR embedder (ours).

Following the analysis protocol in the SNIP supplementary material (Meidani et al., 2024), we evaluate latent organization using the same four function-level attributes: *NCR*, *Upwardness*, *Average Normalized y*, and *Log Oscillations*. For each attribute, we uniformly sample expressions across its value range and visualize the resulting latent geometry with t-SNE. This setup allows us to compare not only whether the spaces are separable, but also how smoothly functional properties align with local neighborhoods.

Reconstruction and Implications for Generative Modeling. We compare reconstruction fidelity across the three spaces. For SNIP variants, we train a 2-layer MLP decoder on each frozen encoder; for MOD-SR, reconstruction is measured natively through diffusion denoising and rounding via the weight-tied output head. The SNIP pooled space achieves only 0.6518 accuracy, confirming that compressing an expression tree into a single vector irreversibly discards token identity—such a space is fundamentally unusable for latent diffusion. In contrast, the SNIP token-wise space reaches 0.9997. As expected, the SNIP token-wise space is viable for latent diffusion and shows competitive early performance, but its peak is comparable to a DiffusionLM trained directly on the token embedding space, while incurring the extra overhead of training a separate decoder (Zheng et al., 2026).

t-SNE Analysis. From Fig. 6, several patterns are immediately apparent. *First*, the two SNIP variants already reveal that removing pooling preserves more useful geometric structure: the token-wise SNIP space is visibly more organized than the pooled one, indicating that compressing an expression into a single vector discards behaviorally relevant information. However, the MOD-SR latent space is more structured still. Rather than forming broad and weakly organized clouds, it breaks into compact and clearly separated clusters. Importantly, this stronger clustering is not

merely a visual artifact of over-segmentation. Within each cluster, samples tend to exhibit much higher attribute consistency than in either SNIP space, suggesting that nearby latent points correspond to expressions with more similar functional behavior. *Second*, MOD-SR provides noticeably clearer separation between extreme attribute values. Across all four rows, the most red and most blue samples are more cleanly isolated in the MOD-SR column than in the SNIP pooled or SNIP token-wise columns, where extremes remain more intermixed with intermediate points. This is especially important from the perspective of symbolic generation: a latent space is most useful not simply when it is smooth, but when meaningful functional differences are reflected in its geometry. MOD-SR appears to achieve a better balance between local coherence and global discriminability.

We believe this property is particularly beneficial for downstream symbolic search and refinement. In a space where clusters are internally coherent and functional extremes are well separated, local moves are more likely to remain within a stable symbolic basin, while transitions across basins correspond to more meaningful changes in behavior. This provides a more reliable geometry for generation and latent-space optimization.

6. Conclusion and Future Work

We presented a framework for symbolic regression that unifies multimodal distribution learning and optimization with gradient-guided diffusion language model, mitigating the training-evaluation misalignment and offering more controllable generation. We also design DFEX to enable gradient guidance for symbolic structure.

Future work targets three axes: (1) accelerated guidance process; (2) applications of fast diffusion-based models (Song et al., 2023) to symbolic regression; (3) extending the framework to ODE/PDE discovery.

Impact Statement

This work provides a practical pathway to more accurate and controllable discovery of interpretable symbolic models by unifying learned generative priors with inference-time optimization. MOD-SR can accelerate scientific equation discovery by reducing dependence on expensive heuristic search, while enabling explicit trade-offs between accuracy and simplicity at inference time. Users should note the additional compute required for gradient-guided inference and balance runtime versus solution quality when deploying the method.

References

- Bastiani, Z., Kirby, R. M., Hochhalter, J., and Zhe, S. Diffusion-based symbolic regression, 2025.
- Biggio, L., Bendinelli, T., Neitz, A., Lucchi, A., and Parascandolo, G. Neural symbolic regression that scales. *CoRR*, abs/2106.06427, 2021.
- Black, K., Janner, M., Du, Y., Kostrikov, I., and Levine, S. Training diffusion models with reinforcement learning. In *The Twelfth International Conference on Learning Representations*, 2024.
- Cao, L., Lin, Z., Tan, K. C., and Jiang, M. Interpretable solutions for multi-physics pdes using t-nngp. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(13): 14212–14220, Apr. 2025.
- Dhariwal, P. and Nichol, A. Q. Diffusion models beat GANs on image synthesis. In Beygelzimer, A., Dauphin, Y., Liang, P., and Vaughan, J. W. (eds.), *Advances in Neural Information Processing Systems*, 2021.
- Fang, W.-Z., Chang, C.-H., Liu, J.-C., and Yu, T.-L. Gp with ranging-binding technique for symbolic regression. In *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*, pp. 563–566, 2023.
- Feng, M., Huang, Y., Liu, Y., Jiang, B., and Yan, J. Physpde: Rethinking pde discovery and a physical hypothesis selection benchmark. In Yue, Y., Garg, A., Peng, N., Sha, F., and Yu, R. (eds.), *International Conference on Representation Learning*, volume 2025, pp. 2043–2065, 2025.
- Guo, Y., Yuan, H., Yang, Y., Chen, M., and Wang, M. Gradient guidance for diffusion models: An optimization perspective. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- Han, X., Ning, C., Zhong, J., Yang, F., Wang, Y., and Mu, X. Discovering mathematical equations with diffusion language model, 2025.
- Hu, J., Cui, J., and Yang, B. Learning interpretable network dynamics via universal neural symbolic regression. *Nature Communications*, 16(1):6226, Jul 2025. ISSN 2041-1723.
- Huynh, P.-T., Bao, F., Yang, H., and Zytoon, A. Finite expression method with trannet-based function learning for high-dimensional partial differential equations, 2026.
- Kamienny, P.-A., d’Ascoli, S., Lample, G., and Charton, F. End-to-end symbolic regression with transformers. *Advances in Neural Information Processing Systems*, 35: 10269–10281, 2022.
- Kamienny, P.-A., Lample, G., Lamprier, S., and Virgolin, M. Deep generative symbolic regression with Monte-Carlo-tree-search. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J. (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 15655–15668. PMLR, 23–29 Jul 2023.
- La Cava, W., Burlacu, B., Virgolin, M., Kommenda, M., Orzechowski, P., de França, F. O., Jin, Y., and Moore, J. H. Contemporary symbolic regression methods and their relative performance. *Advances in neural information processing systems*, 2021(DB1):1, 2021.
- Li, Q., Hu, Y., Liu, J., and Chen, Y. GenSR: Symbolic regression based on equation generative space. In *The Fourteenth International Conference on Learning Representations*, 2026.
- Li, X. L., Thickstun, J., Gulrajani, I., Liang, P., and Hashimoto, T. Diffusion-LM improves controllable text generation. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022.
- Li, Y., Guo, J., Wang, R., and Yan, J. From distribution learning in training to gradient search in testing for combinatorial optimization. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- Li, Y., Guo, J., Wang, R., Zha, H., and Yan, J. Fast t2t: Optimization consistency speeds up diffusion-based training-to-testing solving for combinatorial optimization. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- Li, Y., Li, W., Yu, L., Wu, M., Liu, J., Wei, S., Deng, Y., and Hao, M. Metasymnet: a tree-like symbol network with adaptive architecture and activation functions. AAAI’25/IAAI’25/EAAI’25. AAAI Press, 2025a. ISBN 978-1-57735-897-8.

- Li, Y., Liu, J., Wu, M., Yu, L., Li, W., Ning, X., Li, W., Hao, M., Deng, Y., and Wei, S. Mmsr: symbolic regression is a multi-modal information fusion task. *Information Fusion*, 114:102681, 2025b.
- Liang, S. and Yang, H. Finite expression method for solving high-dimensional partial differential equations. *Journal of Machine Learning Research*, 26(138):1–31, 2025.
- Lin, T. Y., Yao, J., Chiang, L., Berner, J., and Anandkumar, A. Decoupled diffusion solver for inverse problems on function spaces. In *AI&PDE: ICLR 2026 Workshop on AI and Partial Differential Equations*, 2026.
- Lu, Q., Luo, Y., Li, H., Luo, J., and Wang, Z. Deep differentiable symbolic regression neural network. *Neurocomputing*, 629:129671, 2025. ISSN 0925-2312.
- Meidani, K., Shojaei, P., Reddy, C. K., and Farimani, A. B. SNIP: Bridging mathematical symbolic and numeric realms with unified pre-training. In *The Twelfth International Conference on Learning Representations*, 2024.
- Mežnar, S., Džeroski, S., and Todorovski, L. Efficient generator of mathematical expressions for symbolic regression. *Machine Learning*, 112(11):4563–4596, 2023.
- Mojtaba Valipour, Maysum Panju, B. Y. A. G. Symbolicgpt: A generative transformer model for symbolic regression. In *Preprint Arxiv*, 2021. Under Review.
- Mundhenk, T. N., Landajuela, M., Glatt, R., Santiago, C. P., faissol, D., and Petersen, B. K. Symbolic regression via deep reinforcement learning enhanced genetic programming seeding. In Beygelzimer, A., Dauphin, Y., Liang, P., and Vaughan, J. W. (eds.), *Advances in Neural Information Processing Systems*, 2021.
- Oh, H., Amici, R., Bomarito, G., Zhe, S., Kirby, R., and Hochhalter, J. Genetic programming based symbolic regression for analytical solutions to differential equations, 02 2023.
- Peebles, W. and Xie, S. Scalable diffusion models with transformers, 2023.
- Raghav, S. S., Kumar, S. T., Balaji, R., Sanjay, M., and Shunmuga, C. Interactive symbolic regression-a study on noise sensitivity and extrapolation accuracy. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pp. 2076–2082, 2024.
- Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. High-resolution image synthesis with latent diffusion models, 2022.
- Rudy, S. H., Brunton, S. L., Proctor, J. L., and Kutz, J. N. Data-driven discovery of partial differential equations. *Science Advances*, 3(4):e1602614, 2017.
- Sadr, M., Tohme, T., and YUCEF-TOUMI, K. Data-driven discovery of PDEs via the adjoint method. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856.
- Shojaei, P., Meidani, K., Barati Farimani, A., and Reddy, C. Transformer-based planning for symbolic regression. *Advances in Neural Information Processing Systems*, 36: 45907–45919, 2023.
- Shojaei, P., Meidani, K., Gupta, S., Farimani, A. B., and Reddy, C. K. LLM-SR: Scientific equation discovery via programming with large language models. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Simon Keren, L., Liberzon, A., and Lazebnik, T. A computational framework for physics-informed symbolic regression with straightforward integration of domain knowledge. *Scientific Reports*, 13:1249, 01 2023.
- Song, J., Meng, C., and Ermon, S. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021.
- Song, Y., Dhariwal, P., Chen, M., and Sutskever, I. Consistency models, 2023.
- Tenachi, W., Ibata, R., and Diakogiannis, F. I. Deep symbolic regression for physics guided by units constraints: Toward the automated discovery of physical laws. *The Astrophysical Journal*, 959(2):99, December 2023. ISSN 1538-4357.
- Tian, Y., Zhou, W., Dong, H., Kammer, D. S., and Fink, O. Sym-q: Adaptive symbolic regression via sequential decision-making. *arXiv preprint arXiv:2402.05306*, 2024.
- Tymkow, R. T., Schnapp, B. D., Valipour, M., and Ghodshi, A. Symbolic-diffusion: Deep learning based symbolic regression with d3pm discrete token diffusion, 2025.
- Udrescu, S.-M. and Tegmark, M. Ai feynman: a physics-inspired method for symbolic regression, 2020.
- Vafa, K., Chang, P. G., Rambachan, A., and Mullainathan, S. What has a foundation model found? using inductive bias to probe for world models, 2025.
- Yao, J., Mammadov, A., Berner, J., Kerrigan, G., Ye, J. C., Azizzadenesheli, K., and Anandkumar, A. Guided diffusion sampling on function spaces with applications to PDEs. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026.

- Ying, J., Yue, C., Chen, Y., Xiao, C., Shi, Q., Liang, Y., Yau, S.-T., Zhou, Y., and Ma, J. A neural symbolic model for space physics. *Nature Machine Intelligence*, pp. 1–16, 10 2025.
- Yu, S., Kwak, S., Jang, H., Jeong, J., Huang, J., Shin, J., and Xie, S. Representation alignment for generation: Training diffusion transformers is easier than you think. In *International Conference on Learning Representations*, 2025a.
- Yu, Z., Ding, J., Li, Y., and Jin, D. Symbolic regression via MDLformer-guided search: from minimizing prediction error to minimizing description length. In *The Thirteenth International Conference on Learning Representations*, 2025b.
- Zeng, P., Song, X., Lensen, A., Ou, Y., Sun, Y., Zhang, M., and Lv, J. Differentiable genetic programming for high-dimensional symbolic regression. *arXiv preprint arXiv:2304.08915*, 2023.
- Zheng, B., Ma, N., Tong, S., and Xie, S. Diffusion transformers with representation autoencoders. In *The Fourteenth International Conference on Learning Representations*, 2026.

A. Discussions on Conditional Encoders and DFEXHead

A.1. Conditional Encoder for Numerical Observations

We evaluate the performance of the SNIP encoder compared to the E2E encoder for conditioning on numerical observations. Our experiments show that SNIP and E2E encoders perform similarly (Fig. 7). This finding contrasts with DiffuSR’s results (Han et al., 2025), possibly because we use the pre-pooling features from the SNIP encoder for conditioning.

Additionally, we find that the frozen encoder’s effectiveness is heavily influenced by the format of the expressions it was pre-trained on. When we just adjust the float precision of symbolic expressions from 3 to 2 (changing the constant tokens), the training performance significantly degrades compared to float precision 3. This is illustrated in Figure 8, which compares accuracy over epochs.

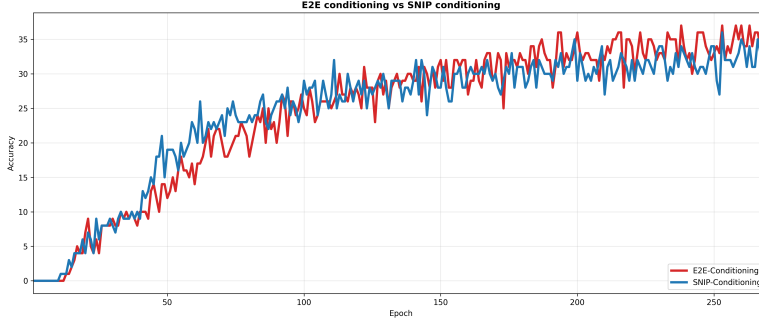


Figure 7. Comparison of E2E-conditioning and SNIP-conditioning.

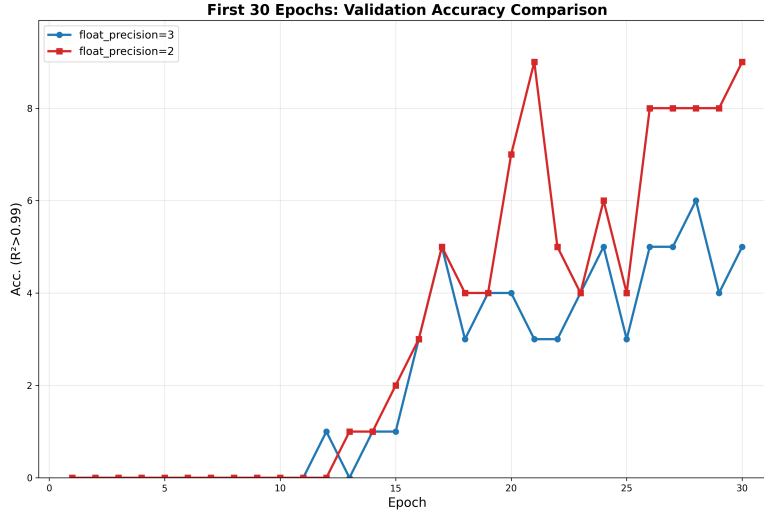


Figure 8. Comparison of accuracy over epochs for float precision 3 vs 2.

A.2. Rationale of Introducing DFEXHead

As mentioned above, the encoder’s effectiveness is heavily influenced by the format of the expressions it was pre-trained on. We found that directly retraining a denoiser using the E2E encoder converges much slower than using the original expression method (and as noted above, this is clearly not an issue with our expression representation method). Moreover, training in the DFEX-style latent space consumes more memory than the original. Therefore, we chose to perform denoising in the original expression form’s embedding space (to facilitate training and reduce inference memory overhead). Then, after training the denoiser, we train an additional DFEXHead, whose input is the embedding space of the trained embedder, and output is the logits for the DFEX-style sequence, where gradients are obtained and backpropagated. During training, we ensure that padding tokens (<PAD>) are not ignored in the attention mechanism to maintain proper sequence processing.

Experiments show that after 100 epochs of training, the DFEXHead’s reconstruction loss reached 0.003, and its performance is completely on par with the original diffusion model.

We also compare the performance of using DFEXHead versus directly training the denoiser in the DFEX-style latent space, all taking 300 epochs. Notably, the accuracy of the non-DFEXHead method doesn’t improve since 150 epochs, while the DFEXHead method continues to improve until the end. The final performance comparison is shown in Table 6. Arguably, this is due to the numerical encoder being pre-trained on the original expression format, making it challenging to adapt to the DFEX-style format.

Table 6. Performance comparison of using DFEXHead vs directly training in DFEX-style latent space.

Dataset	DFEXHead		Direct	
	R^2	C	R^2	C
Nguyen	0.989	15.8	0.980	19.2
Keijzer	0.924	17.6	0.925	14.9
Koza	1.000	10.0	0.933	5.0
Constant	0.982	17.1	0.957	16.4
Livermore	0.955	17.3	0.933	15.1
R	0.994	11.3	0.998	16.0
Avg	0.974	14.8	0.954	14.4

B. Implementation Details

B.1. Network Architecture

Our denoising transformer consists of 128-dimensional token embeddings for a variable vocabulary size, sinusoidal timestep embeddings projected to 128 dimensions, 12 decoder layers each with cross-attention to numerical encoder features (12 heads), FFN with GELU activation (hidden dim 3072), layer normalization ($\epsilon = 1e - 12$), and dropout (0.1), and an output head with linear projection to vocabulary logits with weight tying.

For REPA components, we use a frozen SNIP symbolic encoder with 12 layers and 768 hidden dimensions, a projection MLP from 768 to 768 to 512 with LayerNorm and GELU, and alignment at decoder layer 6.

B.2. Training Details

Hardware Details The embedder, denoiser, and DFEXHead are all trained on 4 A100 GPUs. All inference on a single A100 GPU. Training embedder and denoiser costs approximately 2 days. Training DFEXHead costs approximately 2 hours. Our implementation is also tested on Huawei Ascend NPUs.

Data Generation Expressions are generated using a random function generator that creates symbolic expression trees. The generator randomly constructs multi-dimensional expression trees with configurable numbers of binary and unary operators, input and output dimensions. For each generated expression tree, data points are sampled from a Gaussian mixture or a uniform distribution in the input space, and the corresponding output values are computed by evaluating the expression on these inputs. Invalid points (e.g., those resulting in NaN or infinite values) are filtered out to ensure numerical stability. When training DFEXHead, we still use this generator to create training data. However, we convert the generated expressions into fixed-depth tree format with depth=8 using the algorithm described in Appendix E.

Optimization Table 7 summarizes optimization details for training the embedder, denoiser, and DFEXHead.

Diffusion Schedule Table 8 summarizes the diffusion configuration.

B.3. Gradient Guidance Implementation

We apply guidance selectively during the mid-phase of diffusion where it is most effective. Let $\hat{t} = t/T$ be the normalized timestep. For MSE guidance, activation uses $\hat{t} \in [0.3, 0.7]$ with at most 5 steps; for length guidance, we expand to $\hat{t} \in [0.1, 0.9]$ with up to 30 steps, as structural simplification benefits from earlier and more extensive intervention.

Table 7. Training configurations for Embedder & Denoiser and DFEXHead components.

Component	Embedder & Denoiser	DFEXHead
Optimizer	AdamW	AdamW
Learning rate	5e-5	3e-4
Learning rate schedule	–	CosineAnnealingLR
Weight decay	0.01	1e-4
Gradient clipping	1.0	1.0
Batch size	64	16
Num workers	8	16
Max epochs	300	100
Max epoch size	1000	10000

Table 8. Diffusion schedule configuration.

Parameter	Value
Total timesteps T	2000
Noise schedule $(\beta_{\min}, \beta_{\max})$	$(10^{-4}, 2 \times 10^{-2})$
DDIM sampling steps	50
EMA decay	0.9999

The MSE guidance operates through BFGS-based inner-loop optimization with top- k relaxation and random subtree masking (Section F). The length guidance instead uses autograd-based direct gradient descent on the DFEX logits, penalizing tokens that should be replaced by identity or padding tokens. Both modes share the same DFEX framework but differ in optimizer backend, step budget, and scale. Table 9 summarizes the guidance hyperparameters.

Table 9. Gradient guidance configuration for MSE and length objectives.

Parameter	MSE	Length	Description
d_{sub}	5*	5	Depth of randomly sampled subtree for local optimization
γ	1000	5000	Guidance scale controlling the strength of the gradient update
k	3	–	Number of top candidates kept per position
τ	5.0	2.0	Softmax / guidance temperature
c	20.0	–	Logit clip value applied after row-max normalization
λ_{01}	0.05	–	Weight for the sharpness regularizer during guidance
K_{inner}	5 (BFGS)	1 (Autograd)	Number of inner-loop iterations per guidance step
M	5	30	Maximum number of guidance steps per trajectory
$(t_{\min}, t_{\max})$	(0.3, 0.7)	(0.1, 0.9)	Normalized timestep window for guidance activation

*: 8 for high-dimensional benchmarks like Feynman.

Gradients outside the subtree are zeroed by a mask so that only the targeted nodes move. The complete optimization procedure is shown in Algorithm 1.

Algorithm 1 Optimize Expression via Subtree BFGS

```

1: Input: dataset  $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ , DFEX tree, max iterations  $K$ , top- $k$  size  $k$ 
2: Output: optimized expression  $e^*$ 
3: Initialize: logits  $\ell \in \mathbb{R}^{L \times |\mathcal{V}|}$  (strong init:  $\ell_{target} = 10$ )
4:  $X \leftarrow [x_1, \dots, x_N]$ ,  $Y \leftarrow [y_1, \dots, y_N]$ 
5: for  $iter = 1$  to  $K$  do
6:   Sample subtree  $\mathcal{T}_k \leftarrow \text{SAMPLESUBTREE}(d_{\text{sub}})$  {non-ID subtree}
7:   Build top- $k$  indices  $\mathcal{I}_k \leftarrow \text{TopK}(\ell, k)$  {Restrict optimization space}
8:    $\ell_{\text{topk}} \leftarrow \text{Gather}(\ell, \mathcal{I}_k)$ ;  $\ell_{\text{topk}} \leftarrow \text{Clip}(\ell_{\text{topk}}, -c, c)$ 
9:    $P \leftarrow \text{softmax}(\ell_{\text{topk}}, \tau)$ ;  $M \leftarrow \text{MASK}(\mathcal{T}_k)$   $\{M \in \{0, 1\}^{L \times k}\}$ 
10:   $\hat{y} \leftarrow \text{COMPUTERELAXED}(P \odot M, X)$  {Alg. 3}
11:   $\mathcal{L}_{\text{fit}} \leftarrow \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$ 
12:   $\mathcal{L}_{01} \leftarrow -\frac{1}{k} \sum_{i,j} (P_{ij} - 0.5)^2$ 
13:   $\mathcal{L} \leftarrow \mathcal{L}_{\text{fit}} + \lambda_{01} \mathcal{L}_{01}$ 
14:   $\ell_{\text{topk}} \leftarrow \text{BFGS}(\mathcal{L}, \ell_{\text{topk}})$  {L-BFGS-B on top- $k$  logits}
15:   $\ell \leftarrow \text{Scatter}(\ell, \mathcal{I}_k, \ell_{\text{topk}})$ 
16: end for
17:  $e^* \leftarrow \arg \max_j P_{ij}$  for each position  $i$  {discretize}
18: return  $e^*$ 

```

Length GG The length objective directly optimizes the token-level logits to promote simpler expression structures. Let $\ell \in \mathbb{R}^{L \times |\mathcal{V}|}$ denote the FEX logits. For each active position i where the current top-1 token differs from the target simplicity token (identity, unary, or pad), we minimize the negative logit:

$$\mathcal{L}_{\text{length}} = \frac{1}{|\mathcal{A}|} \sum_{i \in \mathcal{A}} \text{softplus}(-\ell_{i,t(i)}/\tau) \quad (17)$$

where $\mathcal{A}$ is the set of active positions, $t(i)$ maps position i to its target simplicity token based on position type (binary/unary/leaf), and τ is the temperature. The gradient $\nabla_{\ell} \mathcal{L}_{\text{length}}$ is then backpropagated through the DFEX Head to the token embeddings, producing a guidance signal in the original embedding space. During optimization, we recompute the DFEX forward pass to ensure a fresh gradient graph, and apply the resulting signal directly to the predicted clean sample $\hat{x}_0(x_t)$ via autograd-based inner-loop updates with a step count of $K_{\text{inner}} = 10$ per guidance step. We adopt a maximum of $M = 30$ guidance steps per trajectory, with scale $\gamma = 5000$ and temperature $\tau = 2.0$, applied within the normalized timestep window $\hat{t} \in [0.1, 0.9]$, using subtree depth $d_{\text{sub}} = 5$.

MSE GG At each step, we restrict optimization to the top- k candidates ($k = 3$ in our experiments) on the selected subtree. Positions outside the subtree are frozen by setting their softmax probabilities to deterministic one-hot distributions, so guidance never touches the entire vocabulary at once. Given the subtree $\mathcal{T}$, we minimize the masked loss

$$L_{\text{MSE}}(\mathcal{T}) = \frac{1}{|\mathcal{D}|} \sum_{(x_i, y_i) \in \mathcal{D}} (f_{\text{DFEX}}(x_i; \mathcal{T}) - y_i)^2 + \lambda_{01} \|p_{\text{subtree}} - \frac{1}{2}\|_2^2,$$

where p_{subtree} are the softmax logits of the editable subtree. The gradient is masked by M (indicator of $\mathcal{T}$):

$$\nabla_{x_t} L_{\text{MSE}} = \left(M \odot \frac{\partial L_{\text{MSE}}}{\partial p_{\text{subtree}}} \right) \frac{\partial p_{\text{subtree}}}{\partial \hat{x}_0} \frac{\partial \hat{x}_0}{\partial x_t}.$$

Each guided step runs $K = 5$ inner-loop BFGS iterations on $\mathcal{T}$.

This combination of random subtree masking, top- k restriction, temperature-scaled softmax, logit clipping, and inner-loop refinement yields stable and informative MSE guidance.

The auxiliary λ_{01} term with weight 0.05 is implemented exactly as in Algorithm 1: $\mathcal{L}_{01} = \lambda_{01} \|p_{\text{subtree}} - 0.5\|_2^2$. It penalizes indecisive logits for the editable subtree (without touching clamped nodes), encouraging sparse, near-one-hot selections so that gradients stay informative even when we revisit the same subtree multiple times.

B.4. Configurations for Post-processing methods

For DiffuSR, since it’s not open-sourced, we use the same data from its original paper. For SNIP, we use the original configuration in their repository. The whole runtime is approximately 450s for each instance (This is the maximum, and there exists early stop). For our case, we use `rewrite_step=5`, $\alpha = 0.2$, and top-50 sampling to ensure a similar runtime.

Note that we apply the same BFGS post-processing to all methods for fair comparison. The complexity is all calculated upon the simplified expression.

C. Length Gradient Guidance: Detailed Analysis

We present a detailed analysis of the gradient-guided simplicity objective that promotes parsimonious symbolic expressions during inference. Unlike conventional post-processing complexity penalties, our guidance mechanism actively steers the generation process toward simpler structures. Fig. 9 visualizes the predominant simplification patterns observed during guidance.

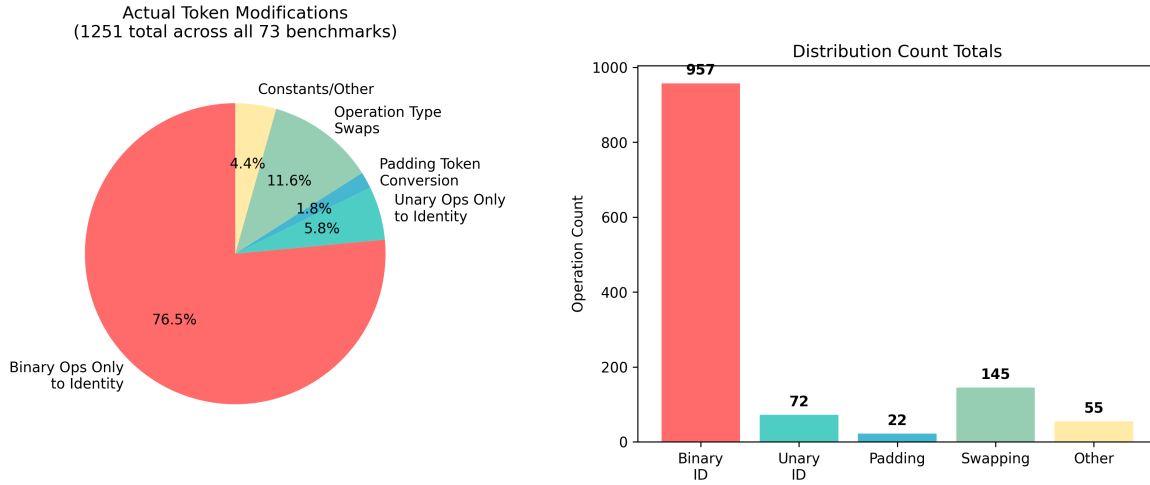


Figure 9. Actual token modifications across all 73 benchmarks (top-20 sampling): 67.7% are binary operations converted to identity placeholders (957 ops), demonstrating systematic structural refinement rather than wholesale mathematical simplification.

We demonstrate actual structural simplification through gradient guidance using the $y = \sqrt{x}$ benchmark:

Simplification Trajectory Analysis From the token modification logs:

Step t=1360: pos 134: mul -> <ID_Binary> → removes multiplication operation

Step t=1320: pos 142: add -> <ID_Binary> → eliminates addition node

Step t=1280: pos 128: N1420 -> PAD → removes constant node

Structural Complexity Reduction The guidance demonstrates clear simplification trends:

Early Phase (steps 16-25): High activity with binary operation removal: 1) 11 mul/add operations converted to identity placeholders, 2) 8 token positions replaced with <PAD> during initial pruning

Peak Simplification (steps 26-35): Maximum gradient activity: 1) Constant simplification: $-N5810 \rightarrow +N1000$ reduces precision digits, 2) Unary operation removal: $\text{abs} \rightarrow \text{<ID_Unary>}$

Convergence (steps 36-49): Structure stabilization: 1) no token change becomes dominant (final 9 steps silent), 2) Tree complexity levels off to minimal functional structure

Algorithm 2 Depth-Aware Tree Encoding

```

1: function FILL( $expr, idx, tree$ )
2:    $node \leftarrow tree[idx]$ 
3:   if  $node.type = \text{LEAF}$  then  $tree[idx] \leftarrow \text{ENCODELEAF}(expr)$ ; return
4:
5:   if  $node.type = \text{UNARY}$  and  $expr$  is unary op then
6:      $n_{\text{avail}} \leftarrow \# \text{Unary layers from } node.layer \text{ to leaf}$ 
7:      $n_{\text{need}} \leftarrow \lceil \text{GetStructuralDepth}(expr)/2 \rceil$ 
8:      $skip \leftarrow \lfloor (n_{\text{avail}} - n_{\text{need}} + 1)/2 \rfloor \{ \text{Midpoint} \}$ 
9:     while  $skip > 0$  do
10:       $tree[idx] \leftarrow \text{ID}; idx \leftarrow \text{CHILD}(idx); skip \leftarrow skip - 1$ 
11:    end while
12:     $tree[idx] \leftarrow \text{op}(expr); \text{FILL}(\text{arg}(expr), \text{CHILD}(idx), tree)$ 
13:  else if  $node.type = \text{UNARY}$  then
14:     $tree[idx] \leftarrow \text{ID}; \text{FILL}(expr, \text{CHILD}(idx), tree)$ 
15:
16:  if  $node.type = \text{BINARY}$  and  $expr$  is binary op then
17:     $tree[idx] \leftarrow \text{op}(expr); L, R \leftarrow \text{SPLIT}(expr)$ 
18:     $\text{FILL}(L, \text{LEFTCHILD}(idx), tree); \text{FILL}(R, \text{RIGHTCHILD}(idx), tree)$ 
19:  else if  $node.type = \text{BINARY}$  then
20:     $tree[idx] \leftarrow \text{ID}; \text{FILL}(expr, \text{LEFTCHILD}(idx), tree)$ 

```

D. Limitations

The inference time of the diffusion model is inherently slower than AR models, though DDIM is used.

Additionally, since denoising is performed in continuous embedding space with rounding to discrete tokens only at the final step, we observe that token identities typically stabilize once $t \lesssim 600$ (out of $T = 2000$ total timesteps). This suggests that a significant portion of the later denoising steps refines continuous embeddings without altering discrete structure, potentially allowing for more efficient sampling with fewer steps.

E. Fixed-Depth Tree Encoding Algorithm

We present the core algorithm for encoding symbolic expressions into fixed-depth FEX-style trees with a *depth-aware placement strategy*. This strategy dynamically determines the optimal vertical placement for operations: for shallow sub-expressions, operations are evenly distributed to maximize extensibility; for deep sub-expressions, operations are placed at the shallowest possible layer to prevent depth overflow. Additionally, we explicitly encode powers $x^2, x^3, x^{-1}, \sqrt{x}$ and inverse trigonometric functions as unary operators to reduce binary tree depth consumption.

Tree Structure. For depth d : Binary(layer 0) $\leftrightarrow$ Unary(layer 1) $\leftrightarrow \dots \leftrightarrow$ Leaf(layer $2d-1$). Unary layers: $\{1, 3, 5, \dots, 2d-3\}$.

2-Token Leaf Encoding. Variables: $(+, x_i)$. Constants: (N_m, E_e) where $c \approx m \times 10^e$.

Example. For $\sin(x_0) + \cos(x_1) \cdot \exp(x_2)$ with depth 4 (Unary layers: 1, 3, 5): $\sin$ branch: $n_{\text{avail}} = 3, n_{\text{need}} = 1, skip = \lfloor (3 - 1 + 1)/2 \rfloor = 1 \Rightarrow$ place at layer 3; $\cos, \exp$ branches: $n_{\text{avail}} = 2, n_{\text{need}} = 1, skip = 1 \Rightarrow$ place at layer 5. Thus, operations are at layers 0, 2, 3, 5 with balanced spacing for extensibility.

F. Differentiable Expression through Relaxation

We present the core relaxation algorithm enabling MSE gradient guidance. Algorithm 3 computes differentiable outputs from softmax probability distributions over token sequences, enabling gradient backpropagation through $\mathcal{L}_{\text{fit}}$. The key difference of our method from DGP (Zeng et al., 2023) is that the fixed tree structure eliminates the adjacency matrix. Our relaxation method can also be understood from a sparse regression point of view (Rudy et al., 2017; Sadr et al., 2025).

Following MetaSymNet (Li et al., 2025a), we employ an entropy-based sharpness penalty $\mathcal{L}_{01} = -\log_2(\max p)$ to encourage peaked distributions during gradient guidance, rather than the simple deviation penalty $\|p - 0.5\|_2^2$.

Note that DDSR-NN (Lu et al., 2025) also claims to be differentiable, but actually, the differentiable part is only in coefficient optimization, not discrete expression structure optimization.

Algorithm 3 Compute Relaxed Expression

```

1: Input: Probabilities  $P \in \mathbb{R}^{L \times |\mathcal{V}|}$ , data  $X \in \mathbb{R}^{N \times d}$ 
2: Output: Predictions  $\hat{y} \in \mathbb{R}^N$ 
3:
4: function COMPUTENODE( $idx$ )
5:   if  $node[idx].type = \text{LEAF}$  then
6:      $p_1, p_2 \leftarrow P[seq\_pos], P[seq\_pos + 1]$  {2-token}
7:      $m \leftarrow \sum_i p_1[N_i] \cdot i$ ,  $e \leftarrow \sum_j p_2[E_j] \cdot j$  {Softmax mixing}
8:      $const \leftarrow m \times 10^e$ 
9:      $sign \leftarrow p_1[+] - p_1[-]$ 
10:     $var \leftarrow sign \times \sum_k p_2[x_k] \cdot X[:, k]$ 
11:    return  $(\sum_i p_1[N_i] \cdot const + (1 - \sum_i p_1[N_i]) \cdot var)$ 
12:
13:   if  $node[idx].type = \text{UNARY}$  then
14:      $c \leftarrow \text{COMPUTENODE}(child\_idx)$ 
15:     return  $\sum_{op \in \text{Unary}} P[seq\_pos][op] \cdot \phi_{op}(c)$  {Weighted ops}
16:
17:   if  $node[idx].type = \text{BINARY}$  then
18:      $l \leftarrow \text{COMPUTENODE}(left\_idx)$ ,  $r \leftarrow \text{COMPUTENODE}(right\_idx)$ 
19:     return  $\sum_{op \in \text{Binary}} P[seq\_pos][op] \cdot \phi_{op}(l, r)$ 
20:
21: return COMPUTENODE( $root\_idx$ )
    
```

G. Noise Robustness Experiment

To evaluate the robustness of MOD-SR to noisy observations, we conducted experiments with additive Gaussian noise at two levels: $\sigma = 0.01$ and $\sigma = 0.1$. We compare the performance of MOD-SR against the SNIP baseline across benchmark datasets.

Tables 10 and 11 present the mean R^2 scores and complexity measures.

Table 10. Noise robustness at $\sigma = 0.01$: Mean R^2 and complexity.

Dataset	SNIP		MOD-SR	
	R^2	$\mathcal{C}$	R^2	$\mathcal{C}$
Nguyen	0.8426	19.67	0.9863	18.50
Keijzer	0.8111	19.67	0.9084	19.33
Koza	0.6698	15.50	0.9999	14.50
Constant	0.8290	20.00	0.9787	18.88
Livermore	0.9227	22.27	0.9373	20.14
R	0.9687	18.33	0.9959	15.67
Jin	0.9266	21.17	0.9954	18.33
Vladislavleva	0.8133	26.00	0.9366	22.75
Avg	0.8480	20.33	0.9673	18.51

Table 11. Noise robustness at $\sigma = 0.1$: Mean R^2 and complexity.

Dataset	SNIP		MOD-SR	
	R^2	$\mathcal{C}$	R^2	$\mathcal{C}$
Nguyen	0.8143	17.83	0.9019	21.25
Keijzer	0.7992	17.50	0.9350	21.25
Koza	0.9956	14.50	0.9893	14.50
Constant	0.8641	17.00	0.9340	22.12
Livermore	0.8649	23.95	0.8521	21.50
R	0.9226	16.00	0.9602	14.33
Jin	0.7803	19.50	0.9823	19.67
Vladislavleva	0.8679	12.25	0.8543	28.25
Avg	0.8636	17.32	0.9261	19.11